

Dynamic Memory Allocation (DMA)

COMPREHENSIVE GUIDE & COMPLETE QUESTION SOLUTIONS

Prepared for Gate / PSU Examination Practice

Q1. malloc() is declared in which header file?

- a) stdio.h
- b) stdlib.h ✓**
- c) string.h
- d) malloc.h

EXPLANATION:

According to the standard C library definition, functions for dynamic memory management such as *malloc()*, *calloc()*, *realloc()*, and *free()* are declared in the `<stdlib.h>` header file. While some older or non-standard compilers also support `<malloc.h>`, `<stdlib.h>` is standard and fully portable.

Q2. Which function initializes all allocated bytes to zero?

- a) malloc()
- b) calloc() ✓**
- c) realloc()
- d) new

EXPLANATION:

The *calloc()* (contiguous allocation) function allocates memory for an array of elements and automatically initializes all bytes in the allocated block to zero. In contrast, *malloc()* leaves the memory uninitialized, meaning it contains garbage values.

Q3. Which function can increase or decrease the size of previously allocated memory?

- a) `calloc()`
- b) `realloc()` ✓**
- c) `free()`
- d) `new`

EXPLANATION:

The **`realloc()`** (reallocation) function alters the size of previously allocated memory blocks without losing existing data (as long as the new size is larger than or equal to the original size). If necessary, it handles moving the data block to a new memory location.

Q4. Which operator is used for dynamic memory allocation in C++?

- a) `malloc`
- b) `alloc`
- c) `new` ✓**
- d) `create`

EXPLANATION:

In C++, the **`new`** operator is built into the language to dynamically allocate memory on the heap. Unlike C function calls like **`malloc()`**, **`new`** automatically determines the correct size for the type and invokes the object's constructor.

Q5. Memory allocated using `new` resides in:

- a) Stack
- b) Heap ✓**
- c) Cache
- d) Register

EXPLANATION:

All dynamically allocated memory—whether through C standard functions (**`malloc`**, **`calloc`**) or C++ operators (**`new`**)—is located in the **Heap** region of memory, which persists until explicitly released or the program ends.

Q6. Local variables are usually stored in:

- a) Heap
- b) Stack ✓**
- c) ROM
- d) Global Memory

EXPLANATION:

Local variables declared inside functions are allocated automatically on the **Stack** frame associated with that function execution. They follow a LIFO structure and are reclaimed when the function scope exits.

Q7. Which statement correctly deallocates an array allocated using new?

- a) delete arr;
- b) delete[] arr; ✓**
- c) free(arr);
- d) remove(arr);

EXPLANATION:

When an array is dynamically allocated using **new T[N]**, it must be cleared using the array version of the delete operator: **delete[] arr;**. This ensures that the destructor is called for every individual element in the array block.

Q8. Which statement is true regarding malloc()?

- a) Calls constructor
- b) Initializes memory
- c) Returns void* ✓**
- d) Calls destructor

EXPLANATION:

The function prototype for **malloc()** is **void* malloc(size_t size);**. It returns a generic, typeless pointer (**void***) pointing to the start of the allocated chunk. It does not invoke constructors or initialize the contents.

Q9. Which statement is true regarding new?

- a) Returns void*
- b) Calls constructor ✓**
- c) Cannot allocate arrays
- d) Exists only in C

EXPLANATION:

The C++ **new** operator allocates raw memory from the heap and then automatically calls the constructor of the class or primitive type to initialize the object, returning a fully typed pointer.

Q10. Memory leak occurs when:

- a) Memory is freed twice
- b) Allocated memory is never released ✓**
- c) Pointer becomes NULL
- d) Stack becomes full

EXPLANATION:

A memory leak occurs when a program allocates memory on the heap but loses all reference pointers to it without freeing it back to the operating system. Over time, memory usage inflates and can exhaust resources.

Q11. Which pointer refers to already deallocated memory?

- a) Null pointer
- b) Dangling pointer ✓**
- c) Wild pointer
- d) Void pointer

EXPLANATION:

A pointer pointing to a memory address that has already been deallocated (via **free()** or **delete**) is known as a **Dangling pointer**. Accessing this pointer leads to undefined behavior.

Q12. Which pointer is never initialized?

- a) Null pointer
- b) Dangling pointer
- c) **Wild pointer** ✓
- d) Smart pointer

EXPLANATION:

A pointer that is declared but not initialized to a valid location or **NULL/nullptr** is known as a **Wild pointer**. It contains an unpredictable garbage address.

Q13. Which is true about nullptr in C++?

- a) **It is a type-safe null pointer** ✓
- b) It is same as malloc
- c) It stores heap address
- d) It deletes memory

EXPLANATION:

Introduced in C++11, **nullptr** is a type-safe keyword of type **std::nullptr_t**. Unlike the macro **NULL** (which is simply the integer 0), **nullptr** cannot be implicitly converted to integral types, eliminating function overloading ambiguities.

Q14. Which memory allocation is generally faster?

- a) Heap allocation
- b) **Stack allocation** ✓
- c) Both always equal
- d) Depends only on RAM size

EXPLANATION:

Stack allocation is extremely fast because it only requires shifting the stack pointer register (a simple CPU instruction). Heap allocation involves searching complex data structures for free memory blocks, handling fragmentation, and managing locks in multithreaded systems.

Q15. Which memory is automatically reclaimed after a function returns?

- a) Heap
- b) Stack ✓**
- c) ROM
- d) Hard disk

EXPLANATION:

When a function returns, its entire execution stack frame is popped off, which completely and automatically reclaims all stack-allocated variables instantly.

Q16. What happens if delete is applied twice to the same pointer?

- a) Guaranteed correct output
- b) Undefined behavior ✓**
- c) Compile-time success with no issue
- d) Memory automatically doubles

EXPLANATION:

Deallocating the same heap address twice (a double-free error) causes **Undefined Behavior**. It corrupts the allocator's internal structures and can lead to severe runtime crashes or security vulnerabilities.

Q17. Which memory region commonly suffers from fragmentation?

- a) Heap ✓**
- b) Stack
- c) CPU register
- d) ROM

EXPLANATION:

Because memory blocks on the **Heap** are allocated and freed in arbitrary order and sizes, it naturally develops gaps and non-contiguous free spaces over time, leading to both internal and external fragmentation.

Q18. Which fragmentation occurs because free memory is scattered in small blocks?

- a) Internal fragmentation
- b) External fragmentation ✓**
- c) Stack fragmentation
- d) Cache fragmentation

EXPLANATION:

External fragmentation occurs when there is enough total free space available in the system to satisfy an allocation request, but the space is divided into small, non-contiguous chunks scattered across the heap space, preventing any single block from being allocated.

Q19. Which fragmentation occurs due to unused space inside an allocated block?

- a) External fragmentation
- b) Internal fragmentation ✓**
- c) Paging error
- d) Stack overflow

EXPLANATION:

Internal fragmentation happens when memory blocks are allocated in fixed-size chunks (e.g., rounded up to power-of-two boundaries). The difference between the exact space requested by the program and the larger block assigned to it is wasted inside that block.

Q20. Which of the following uses dynamic memory internally?

- a) vector
- b) string
- c) list
- d) All of these ✓**

EXPLANATION:

Standard Template Library (STL) containers like `std::vector`, `std::string`, and `std::list` are designed to automatically scale and hold variable amounts of elements. To support this growth, they manage memory dynamically on the heap under the hood.

Q21. Stack follows which allocation strategy?

- a) FIFO
- b) LIFO ✓**
- c) Random
- d) Circular

EXPLANATION:

The runtime memory **Stack** operates strictly on a **Last-In, First-Out (LIFO)** policy. The variable allocated last inside a function block will be the first one to be popped and destroyed when execution finishes.

Q22. Heap memory generally supports:

- a) Compile-time fixed allocation
- b) Dynamic allocation ✓**
- c) Only LIFO allocation
- d) Only global variables

EXPLANATION:

Unlike the stack or static data segments, **Heap memory** supports completely dynamic, runtime-determined allocation where lifetimes of memory chunks can outlive the scope that created them.